

SupplyChainX Frontend Deep Code Guide

Executive summary

SupplyChainX Frontend is an Angular 20 standalone-component application that implements a role-based operational UI for six personas:

- Admin
- Planner
- Logistics
- Executive
- Procurement
- Warehouse

The app is organized around:

1. **Role-scoped dashboard routes** under a shared authenticated layout.
2. **Domain services** mapping directly to backend REST resources under `https://localhost:7295/api/`.
3. **Strong DTO model contracts** for user, network master data, orders, shipments, inventory, exceptions, resolutions, KPIs, and notifications.
4. **Cross-cutting auth/error interceptors** and **guards/resolvers** for access control and context shaping.
5. **Global UX systems** for toast notifications and in-header notification center polling.

The project uses Angular standalone APIs (`bootstrapApplication` , standalone components, `provideRouter` , `provideHttpClient`) with strict TypeScript and strict Angular template checks enabled.

Full architecture map

Top-level technical architecture

- **Presentation Layer**
 - Standalone components in `src/app/components/**`
 - Shared layout (`main-layout` , `header` , `sidebar`)
 - Feature UIs by role domain
- **Routing Layer**
 - Central route graph in `app.routes.ts`
 - Route guards (`authGuard` , `RoleGuard`)
 - Route resolvers (`OrderContextResolver` variants)
- **Application Services Layer**
 - Auth/session (`authentication.service` , `login.service`)
 - Domain APIs (`order` , `inventory` , `shipment` , `kpi` , etc.)
 - UI state/event services (`toast` , `order-context` , `notification-event`)
- **Infrastructure Layer**
 - HTTP interceptors (`auth` , `error`)
 - App providers in `app.config.ts`
- **Model/DTO Layer**
 - Domain interfaces under `models/*.ts`
 - Shared enums in `enums.ts`
- **Configuration Layer**

- Angular CLI build/test setup (`angular.json`)
- TS strictness/path aliases (`tsconfig*.json`)
- API base URL (`app-settings.ts`)

Feature map by business area

- **Identity & Access**
 - `login screen`
 - `authentication.service` , `login.service`
 - `auth.guard` , `role.guard` , `auth.interceptor`
 - **Admin**
 - User management, user creation, audit logs, network masters (locations/lanes/partners)
 - **Planner**
 - Exception event lifecycle + resolution action lifecycle
 - **Logistics**
 - Dispatch orders + shipment tracking/status progression
 - **Procurement**
 - View locations/partners + constrained order placement context
 - **Warehouse**
 - Inventory management + UOM/item master + constrained order placement context
 - **Executive**
 - KPI summary/trends/risks/reports and chart visualizations
 - **Cross-cutting**
 - Toast system
 - Notification center (poll + read/clear/delete)
-

Angular bootstrap/runtime flow

1. `main.ts`
 - Registers locale data (`en-IN`).
 - Bootstraps root standalone component `App` with `appConfig` .
2. `app.config.ts`
 - Sets `LOCALE_ID` to `en-IN`
 - Sets default currency `INR`
 - Enables zone event coalescing
 - Registers router with `routes`
 - Registers HTTP client with interceptor chain:
 - `authInterceptor`
 - `errorInterceptor`
3. `App` + `app.html`
 - Root view renders:
 - global `app-toast-container`
 - root `router-outlet`

- This ensures toast UI exists across all routes.

4. Runtime navigation

- Public route: `/login`
- Protected app shell: `/` -> `MainLayoutComponent` guarded by `authGuard`
- Child role routes guarded by `RoleGuard`

5. HTTP runtime

- All HTTP calls pass through auth/error interceptors.
- Non-login calls get bearer token injection when available.
- Error interceptor enriches errors with `userMessage`.

Routing flow with role-wise route tree

Global route graph

- `/login` -> `LoginComponent`
- `/` -> `MainLayoutComponent` (`authGuard`)
 - `/admin-dashboard` (`RoleGuard` , roles: `Admin`)
 - `/planner-dashboard` (`RoleGuard` , roles: `Planner`)
 - `/logistics-dashboard` (`RoleGuard` , roles: `Logistics`)
 - `/executive-dashboard` (`RoleGuard` , roles: `Executive`)
 - `/procurement-dashboard` (`RoleGuard` , roles: `Procurement`)
 - `/warehouse-dashboard` (`RoleGuard` , roles: `Warehouse`)
 - default redirect to `/admin-dashboard`
- `/404` -> `NotFoundComponent`
- `**` -> redirect `/404`

Role-wise route tree

Admin

- `/admin-dashboard` -> shell (`AdminDashboardComponent`)
 - `'` -> `AdminDashboardContentComponent` (lazy)
 - `/profile` -> `ViewProfileComponent`
 - `/users` -> `AdminManageUsersComponent` (lazy)
 - `/audit-logs` -> `AdminAuditLogsComponent` (lazy)
 - `/create-user` -> `AdminCreateUserComponent` (lazy)
 - `/manage-network` -> `AdminManageNetworkComponent` (lazy)

Planner

- `/planner-dashboard`
 - `'` -> `PlannerDashboardOverviewComponent` (lazy)
 - `/profile` -> `ViewProfileComponent`
 - `/exceptions` -> `PlannerManageExceptionEventsComponent` (lazy)
 - `/resolutions` -> `PlannerManageResolutionActionsComponent` (lazy)

Logistics

- `/logistics-dashboard`
 - `''` -> redirect `/orders`
 - `/profile` -> `ViewProfileComponent`
 - `/orders` -> `ViewOrdersComponent` + `OrderContextResolver`
 - `/dispatch` -> `LogisticsDispatchOrdersComponent` (lazy)
 - `/shipments` -> `LogisticsManageShipmentsComponent` (lazy)

Executive

- `/executive-dashboard`
 - `/profile` -> `ViewProfileComponent`
 - default section behavior handled via query params (`section`) inside component logic

Procurement

- `/procurement-dashboard`
 - `''` -> `ViewOrdersComponent` + `ProcurementOrderContextResolver`
 - `/profile` -> `ViewProfileComponent`
 - `/view-locations` -> `ProcurementViewLocationsComponent` (lazy)
 - `/view-partners` -> `ProcurementViewPartnersComponent` (lazy)
 - `/orders` -> `AddOrderComponent` + `ProcurementOrderContextResolver`

Warehouse

- `/warehouse-dashboard`
 - `''` -> `WarehouseManageInventoryComponent` (lazy)
 - `/profile` -> `ViewProfileComponent`
 - `/inventory` -> `WarehouseManageInventoryComponent`
 - `/view-orders` -> `ViewOrdersComponent` + `OrderContextResolver`
 - `/place-order` -> `AddOrderComponent` + `WarehouseOrderContextResolver`
 - `/add-uom` -> `WarehouseAddUomComponent` (lazy)
 - `/add-item` -> `WarehouseAddItemComponent` (lazy)

Authentication/authorization flow (guards, interceptors, token handling)

Login and token session

- `login.component` validates email/password, calls `LoginService.login()` .
- `LoginService` posts to `/api/User/Login` expecting plain text JWT.
- JWT is decoded client-side to extract:
 - `userId`
 - `email`
 - `displayName`
 - `role`
- `AuthenticationService` stores:
 - `authToken` in `localStorage`
 - `currentUser` in `localStorage`
 - current user in `BehaviorSubject`

Authorization checks

- `AuthGuard` :
 - allows only when user session exists with role
 - else redirects to `/login?returnUrl=...`
- `RoleGuard` :
 - reads route `data.roles`
 - allows only if authenticated user has one required role
 - otherwise redirects to user's own dashboard by role mapping

HTTP interceptor behavior

- `authInterceptor` :
 - skips URLs containing `/Login`
 - injects `Authorization: Bearer <token>`
 - on 401: logout + redirect to login
- `errorInterceptor` :
 - maps status to user-friendly message (`userMessage`)
 - handles connectivity, 4xx, 5xx classes

Token handling details

- No refresh-token mechanism in frontend.
- Session persistence is localStorage-based.
- Logout clears storage and BehaviorSubject state.

Layout flow (root, main layout, header, sidebar, toast, notification center)

1. App root always renders toast container + router outlet.
2. Authenticated routes render `MainLayoutComponent` .
3. `MainLayoutComponent` composes:
 - `HeaderComponent`
 - `SidebarComponent`
 - inner role feature outlet
4. `HeaderComponent` :
 - displays current user name/role from auth service
 - provides profile navigation based on current URL prefix
 - hosts `NotificationCenterComponent`
 - logout action
5. `SidebarComponent` :
 - computes nav links from role-based link map
 - updates active link using route + query param match
 - supports logout
6. `ToastContainerComponent` :
 - subscribes to `ToastService.toasts$`
 - displays auto-dismiss + manual dismiss alerts
7. `NotificationCenterComponent` :
 - loads current user notifications
 - polls every 30 seconds
 - subscribes to in-app update stream

- supports read, read-all, delete, clear-all

Component-wise explanation (all

`src/app/components/**/*.*.component.ts`)

Notes:

- “Inputs/State” includes `@Input` , forms, and key internal state.
- “Services Used” is exact where explicit; if none, “None”.
- “Key Methods/Logic” is factual from code; if inferred, marked as inferred from naming.

Admin dashboard group

Component Path	Main Responsibility	Inputs/State	Services Used	Key Methods
<code>src/app/components/admin-dashboard/admin-dashboard.component.ts</code>	Admin route shell container	No local state	None	Structure for child components (inferred from naming usage)
<code>src/app/components/admin-dashboard/admin-dashboard-content/admin-dashboard-content.component.ts</code>	Admin landing stats and user snapshot	<code>dashboardStats</code> , <code>recentUsers</code> , <code>isLoading</code>	<code>ManageUserService</code>	<code>loadDashboardStats</code> , <code>loadUserBadges</code>
<code>src/app/components/admin-dashboard/admin-manage-users/admin-manage-users.component.ts</code>	User listing/search/edit	<code>users</code> , <code>filteredUsers</code> , <code>reactive editForm</code> , <code>role options</code>	<code>ManageUserService</code> , <code>ToastService</code>	<code>loadUsers</code> , <code>filterUsers</code> , <code>startEditForm</code> , <code>saveUser</code> , <code>edit flow</code>
<code>src/app/components/admin-dashboard/admin-audit-logs/admin-audit-logs.component.ts</code>	Audit log view/filter/export	<code>auditLogs</code> , <code>filters</code> , <code>date range</code> , <code>unique action list</code>	<code>ManageUserService</code> , <code>ToastService</code>	<code>loadAuditLogs</code> , <code>applyFilters</code> , <code>resetFilters</code> , <code>download</code>
<code>src/app/components/admin-dashboard/admin-create-user/admin-create-user.component.ts</code>	New user registration by admin	Reactive <code>createUserForm</code> , <code>dynamic users</code> , <code>role map</code>	<code>UserService</code> , <code>ManageUserService</code> , <code>ToastService</code> , <code>Router</code>	<code>role-id</code> from <code>existing users</code> , or <code>create form</code> , <code>resetForm</code>
<code>src/app/components/admin-dashboard/admin-manage-network/admin-manage-network.component.ts</code>	Tab host for lanes/locations/partners	<code>activeTab</code>	None	<code>setTab</code>

src/app/components/admin-dashboard/admin-manage-network/manage-network-locations/manage-network-locations.component.ts	Location CRUD orchestration	locations, loading/form/editing flags	LocationService, ToastService	loadLocations saveLocations createLocation branchLocations deleteLocations
src/app/components/admin-dashboard/admin-manage-network/manage-network-locations-list/manage-network-locations-list.component.ts	Location table/list actions	locations, loading	None	Emits events events confirm
src/app/components/admin-dashboard/admin-manage-network/manage-network-locations-form/manage-network-locations-form.component.ts	Location form for create/edit	location, isEditing, reactive form	None	ngOnCharacter patch/reject onSubmit createLocation DTO emit
src/app/components/admin-dashboard/admin-manage-network/lanes/manage-network-lanes.component.ts	Lane CRUD orchestration + location lookup	lanes, locations, form/edit flags	LaneService, LocationService, ToastService	loadLanes loadLocations saveBranches delete, location name match
src/app/components/admin-dashboard/admin-manage-network/lanes-list/manage-network-lanes-list.component.ts	Lane list actions	lanes, locations, loading	None	emit events getLocations
src/app/components/admin-dashboard/admin-manage-network/lanes-form/manage-network-lanes-form.component.ts	Lane create/edit form	lane, locations, reactive form	None	ngOnCharacter onSubmit create/update
src/app/components/admin-dashboard/admin-manage-network/partners/manage-network-partners.component.ts	Partner CRUD orchestration	partners, form/edit flags	PartnerService, ToastService	loadPartners savePartners createLocation close form
src/app/components/admin-dashboard/admin-manage-	Partner list edit endpoint	partners, loading	None	Emits events

network/manage-network-partners/manage-network-partners-list/manage-network-partners-list.component.ts				
src/app/components/admin-dashboard/admin-manage-network/manage-network-partners/manage-network-partners-form/manage-network-partners-form.component.ts	Partner create/edit form + status enum conversion	partner, reactive form	None	ngOnChar onSubmit parseSta

Planner dashboard group

Component Path	Main Responsibility	Inputs/State	Services Used	Metl
src/app/components/planner-dashboard/planner-dashboard.component.ts	Planner route shell	No local state	None	Structu for chi
src/app/components/planner-dashboard/planner-dashboard-overview/planner-dashboard-overview.component.ts	Planner KPI cards from exception/resolution counts	cards, totals, recent exceptions	ExceptionEventService, ResolutionActionService	paralle loading cards i buildDa
src/app/components/planner-dashboard/planner-manage-exception-events/planner-manage-exception-events.component.ts	Exception event CRUD/filter	exceptionEvents, filter/edit forms, enums/options	ExceptionEventService, ToastService	loadExc applyFi openCre openEdi saveExc referer norma
src/app/components/planner-dashboard/planner-manage-resolution-actions/planner-manage-resolution-actions.component.ts	Resolution action CRUD/filter + owner assignment	resolutionActions, exceptions, users, forms	ResolutionActionService, ExceptionEventService, ToastService	role-ta filterin except create/ closed enforc edit

Logistics dashboard group

Component Path	Main Responsibility	Inputs/State	Services Used	Key Methods/Logi
----------------	---------------------	--------------	---------------	------------------

src/app/components/logistics-dashboard/logistics-dashboard.component.ts	Logistics shell	No local state	None	Child route container
src/app/components/logistics-dashboard/logistics-dispatch-orders/logistics-dispatch-orders.component.ts	Dispatch eligible orders into shipments	pendingOrders, partners, dispatchForm, modal state	OrderService, PartnerService, ToastService, FormBuilder	partner filtering by order type, modal dispatch flow, shipment dispatch DTO creation
src/app/components/logistics-dashboard/logistics-manage-shipments/logistics-manage-shipments.component.ts	Shipment tracking, filter, deliver/edit status	shipments, filteredShipments, details modal, status modal	ShipmentService, ToastService, DatePipe	client filters, updateShipmentStatus, deliverShipment, status normalization and class mapping

Procurement dashboard group

Component Path	Main Responsibility	Inputs/State	Services Used	Key Methods/Logic
src/app/components/procurement-dashboard/procurement-dashboard.component.ts	Procurement shell	No local state	None	Child route host
src/app/components/procurement-dashboard/procurement-view-locations/procurement-view-locations.component.ts	Read-only location catalog view	locations, loading/error	LocationService	load + type badge/count/label helpers
src/app/components/procurement-dashboard/procurement-view-partners/procurement-view-partners.component.ts	Read-only partner catalog view	partners, loading/error	PartnerService, ToastService	load + status/type class/count helpers

Warehouse dashboard group

Component Path	Main Responsibility	Inputs/State	Services Used	Key Methods/Log
src/app/components/warehouse-dashboard/warehouse-dashboard.component.ts	Warehouse shell	No local state	None	Child route host
src/app/components/warehouse-dashboard/warehouse-manage-inventory/warehouse-manage-inventory.component.ts	Inventory query + adjust + create position	inventory, filters, forms, selectedInventory	InventoryService, ItemService, LocationService, ToastService	list/filter inventory, low-stock logic from reorder level, adjust/create via InventoryService

src/app/components/warehouse-dashboard/warehouse-add-uom/warehouse-add-uom.component.ts	UOM CRUD with search	uoms, filteredUoms, edit/create flags	UomService, ToastService	list, filter, create/update/delete UOM
src/app/components/warehouse-dashboard/warehouse-add-item/warehouse-add-item.component.ts	Item CRUD with search/detail modal	items, filteredItems, uoms, selected/edit state	ItemService, UomService, ToastService	create/update/delete item, UOM lookup filtered listing

Executive dashboard group

Component Path	Main Responsibility	Inputs/State	Services Used	Me
src/app/components/executive-dashboard/executive-dashboard.component.ts	Executive dashboard composition + section control	activeSection, kpiData, riskData, trendData	ExecutiveDashboardService, KpiService, ActivatedRoute	que sec swi mo loa
src/app/components/executive-dashboard/executive-kpi-summary/executive-kpi-summary.component.ts	KPI summary presentational block	@Input() kpiSummary	None	Pur con
src/app/components/executive-dashboard/executive-kpi-trends/executive-kpi-trends.component.ts	Trends filtering/loading and child chart composition	trendData, date filters, loading state	KpiService	vali ran filt syn filt
src/app/components/executive-dashboard/executive-kpi-trends/components/otif-line-chart.component.ts	OTIF vs delay line chart	@Input() trendData	None (Chart.js direct)	cha ren life res cha
src/app/components/executive-dashboard/executive-kpi-trends/components/inventory-turns-bar-chart.component.ts	Top inventory turns horizontal bar chart	@Input() trendData	None (Chart.js direct)	cha ren life too for
src/app/components/executive-dashboard/executive-risk-metrics/executive-risk-metrics.component.ts	Risk metrics interpretation	@Input() riskSummary	None	shc calc que leve ma

src/app/components/executive-dashboard/executive-reports/executive-reports.component.ts	KPI report list/generate/download	reports, scope, generate/load flags	KpiService	load dup rep har exp
---	--------------------------------------	---	------------	----------------------------------

Shared/common group

Component Path	Main Responsibility	Inputs/State	
src/app/components/login/login.component.ts	User sign-in UX and role redirect	reactive loginForm, loading/error/success, returnUrl	Log Act
src/app/components/shared/layout/main-layout/main-layout.component.ts	Authenticated app frame composition	No local state	Nc
src/app/components/shared/layout/header/header.component.ts	Topbar identity, profile nav, logout	userName, userRole	Aut Rol
src/app/components/shared/layout/sidebar/sidebar.component.ts	Role-based navigation menu	navLinks, role-link map	Aut Rol
src/app/components/shared/toast-container/toast-container.component.ts	Global toast rendering	toasts	Toa
src/app/components/shared/notification-center/notification-center.component.ts	In-app notification inbox	notifications, unreadCount, modal/loading states	Not Aut
src/app/components/shared/not-found/not-found.component.ts	404 view	none	Nc
src/app/components/shared/view-profile/view-profile.component.ts	Current user profile card	currentUser, loading/messages	Aut
src/app/components/shared/view-orders/view-orders.component.ts	Generic orders browser with context-aware visibility	orders, filters, visibleOrderTypes, selected row	Orc Par Ita Orc Toa

src/app/components/shared/add-order/add-order.component.ts	Generic order creation form with context constraints	reactive orderForm + FormArray lines, disabled types	OrderItemPartialLocationOrderTotal
--	--	--	------------------------------------

Service-wise explanation (src/app/services/*.ts)

Service	API Base	Main Methods	Typical Consume
AuthenticationService	None (localStorage/session state)	setCurrentUser, isLoggedIn, hasRole, logout	guards, header, sidebar, login, notification
LoginService	AppSettings.apiEndpoint + 'User'	login, JWT decode/extract helpers, session set	LoginComponent
UserService	AppSettings.apiEndpoint + 'user'	getAllUsers, loginUser, createUser	AdminCreateUserComponent (role mapping + create)
ManageUserService	... + 'ManageUsers', ... + 'NewUserRegistration'	getAllUsers, getUserById, getAuditLogs, editUser, registerUser, deleteUser	admin dashboard manage users, audit
OrderService	... + 'orders' and ... + 'shipments'	placeOrder, getOrder, listOrders, dispatchOrder, deliverShipment	add/view orders, list shipments
ShipmentService	... + 'shipments'	getShipment, listShipments, dispatchShipment, deliverShipment, updateShipment	logistics dispatch/manage shipments
InventoryService	... + 'inventory'	listInventory, adjustInventory, createInventoryPosition	warehouse inventory
ItemService	... + 'items'	createItem, getItem, updateItem, listItems	warehouse item management orders, inventory
UomService	... + 'uoms'	createUom, getUom, updateUom, listUoms	warehouse UOM/management
LocationService	... + 'locations'	createLocation, getLocation, updateLocation, listLocations	admin network location procurement view order, inventory
PartnerService	... + 'partner'	createPartner, getPartner, updatePartner, listPartners	admin network partner procurement view logistics dispatch, order
LaneService	... + 'lanes'	createLane, getLane, updateLane, listLanes	admin network lane

ExceptionEventService	... + 'exceptionevent' (fallback plural endpoint for update)	list/get/filter/create/update/delete exception events	planner overview exception mgmt
ResolutionActionService	... + 'resolutionaction' (fallback plural for owners)	list/get/filter/create/update/delete resolution actions, getAssignableUsers	planner overview resolution mgmt
KpiService	... + 'kpi'	generateReport, viewReportList, getRiskSummary, getKpiTrends, downloadCsv	executive dashboard/trends
ExecutiveDashboardService	Composes KpiService	loadDashboardData via forkJoin, report helper methods	executive dashboard/report
NotificationEventService	... + 'notifications'	create/list/read/read- all/delete/delete-all/ack/update- status + local update stream	notification center potential future ei
OrderContextService	None (UI state only)	set/clear disabled and visible order types	order-context res add-order, view-c
ToastService	None (UI state only)	success, error, info, remove	most feature com for feedback

Guard/resolver explanation (src/app/guards)

- **auth.guard.ts** (CanActivateFn)
 - Enforces authenticated session at main layout entry.
 - Redirects to login with return URL.
- **role.guard.ts** (CanActivate , CanActivateChild)
 - Enforces role membership from route metadata.
 - Redirects unauthorized user to their own dashboard mapping.
- **order-context.resolver.ts**
 - OrderContextResolver : default visibility (PO , SO , Transfer) no disabled types.
 - ProcurementOrderContextResolver : disables SO , shows PO + Transfer .
 - WarehouseOrderContextResolver : disables PO , shows SO + Transfer .

This is a clean pattern for **route-driven UI policy** without hardcoding per-component route checks.

Interceptor explanation (src/app/interceptors)

- **auth.interceptor.ts**
 - Appends bearer token to requests except login path (/Login).
 - Handles unauthorized responses by clearing session and redirecting login.
- **error.interceptor.ts**

- Converts technical HTTP failures into user-facing messages.
 - Adds `userMessage` to thrown error object.
 - Covers status cases: 0, 400, 401, 403, 404, 500, 503, default fallback.
-

Model-wise explanation (`src/app/models`)

Purpose summary by file

- `login.model.ts` : login payload/response, user-role type, role-dashboard constants.
- `app-user.model.ts` , `create-user.model.ts` : user profile and registration DTOs.
- `audit-log.model.ts` : admin audit history row shape.
- `enums.ts` : shared enums (status, severity, type, partner type, etc.).
- `location.model.ts` , `partner.model.ts` , `lane.model.ts` : network master entities.
- `item.model.ts` , `uom.model.ts` : product and UOM masters.
- `inventory.model.ts` : inventory positions and adjust movement DTOs.
- `order.model.ts` : order headers/lines create + response contracts.
- `shipment.model.ts` : dispatch/delivery + shipment response contracts.
- `exception-event.model.ts` : exception event create/update and response contracts.
- `resolution-action.model.ts` : planner action assignment and lifecycle contracts.
- `kpi.model.ts` : KPI/risk/trend request-response DTOs for executive analytics.
- `notification-event.model.ts` : notification CRUD/status/ack contracts.
- `index.ts` : barrel export for model aggregation.

Key DTO families (concise)

1. **Identity DTOs**: login + user.
 2. **Master data DTOs**: location/partner/lane/item/uom.
 3. **Execution DTOs**: order/shipment/inventory.
 4. **Control tower DTOs**: exception/resolution.
 5. **Analytics DTOs**: KPI/risk/trends.
 6. **Communication DTOs**: notifications.
-

Settings/config explanation

`main.ts`

- Entry point.
- Locale registration (`en-IN`), standalone bootstrap.

`app.ts`

- Root component, imports `RouterOutlet` + global `ToastContainerComponent` .

`app.config.ts`

- Global DI providers:
 - locale and currency defaults
 - router
 - HTTP + interceptor chain

app.routes.ts

- Central role-based route tree with lazy `loadComponent` .
- Auth guard at layout level.
- Role guard at dashboard level.
- Resolver-based order context for specific flows.

tsconfig.json

- Strict TypeScript + strict Angular templates.
- Path aliases:
 - `@app/*`
 - `@services/*`
 - `@models/*`
 - `@components/*`

tsconfig.app.json

- App compilation output to `out-tsc/app` .
- Excludes specs.

tsconfig.spec.json

- Test compilation output to `out-tsc/spec` .
- Includes Jasmine types and spec files.

angular.json

- Angular build system `@angular/build:application` .
 - Browser entry `src/main.ts` .
 - SCSS styling and `public/` assets.
 - Production budgets + output hashing.
 - Dev/prod serve targets.
 - Karma test target using `zone.js/testing` .
-

End-to-end flows

1) Login flow

1. User opens `/login` .
2. `LoginComponent` validates form.
3. `LoginService` posts credentials to `/api/User/Login` .
4. JWT (text) decoded client-side; user claims extracted.
5. `AuthenticationService` persists session.
6. Redirect route selected by:
 - `returnUrl` if provided
 - else role dashboard mapping.

2) Place order flow (shared Add Order)

1. Resolver sets order context per route (procurement/warehouse/default).

2. `AddOrderComponent` subscribes to disabled order types.
3. User selects order type; component applies dynamic validators:
 - `PO` : partner + destination required
 - `SO` : origin required
 - `Transfer` : origin + destination required (must differ)
4. Order lines built through `FormArray` .
5. Submit -> `OrderService.placeOrder` -> success toast + form reset.

3) Dispatch shipment flow (Logistics)

1. `LogisticsDispatchOrdersComponent` loads pending orders + partners.
2. Dispatch modal opened for selected order.
3. Partner list filtered by order type (`Supplier/Carrier/3PL`).
4. Submit dispatch form -> `OrderService.dispatchOrder` .
5. On success:
 - success toast
 - remove order from pending list
 - modal closes.

4) Warehouse inventory flow

1. `WarehouseManageInventoryComponent` loads inventory + items + locations.
2. Filters (location/item/low stock/search) call `listInventory` .
3. Low stock is computed from `availableQty <= reorderLevel` .
4. Adjust flow:
 - selects existing position
 - posts `InventoryAdjustDto` to `adjustInventory` .
5. Create flow:
 - location + item + initial qty
 - currently also uses `adjust` endpoint to initialize inventory.

5) Planner exception lifecycle

1. Planner lists/filters exceptions via `ExceptionEventService` .
2. Create/edit form maps to `ExceptionEventUpsertDTO` .
3. Save calls create or update API.
4. Delete confirms then removes event.
5. Status badge helper maps open/in-progress/closed visual style.

6) Planner resolution action lifecycle

1. Planner loads exceptions + assignable users + actions.
2. Owner dropdown filtering rule:
 - Delay -> Logistics users
 - Capacity/Shortage -> Warehouse users
3. Create action defaults status to in-progress.
4. Edit action enforces closed status in component logic.
5. Save calls create/update and refreshes list.

7) Executive KPI flow

1. Dashboard initial load uses `ExecutiveDashboardService` .
2. Service builds requests and runs KPI/risk/trend calls in parallel via `forkJoin` .
3. Section switching via `?section=` query param.
4. Trends child can run custom filtered fetches via `KpiService` .
5. Charts render through Chart.js components.
6. Reports section:
 - list reports by scope
 - generate report
 - export single report CSV.

8) Notifications flow

1. Header hosts `NotificationCenterComponent` .
 2. On init, obtains current user ID from auth service.
 3. Loads notifications (`getMyNotifications`), normalizes incoming ID keys.
 4. Starts 30-second polling.
 5. Subscribes to service-side update stream (`notificationUpdates$`) for in-app pushes.
 6. Supports:
 - mark one read
 - mark all read
 - delete one
 - clear all
 7. UI tracks unread count and unread-first sorting.
-

Important design decisions and strengths

1. **Standalone Angular architecture**
 - No NgModule overhead for feature components.
 2. **Role-first route model**
 - Clear ownership boundaries between personas.
 3. **Lazy loading for feature screens**
 - Better startup behavior and route-level code split.
 4. **Guard + resolver combination**
 - Security and UI context are declaratively route-driven.
 5. **Service-per-domain pattern**
 - Predictable backend API mapping and maintainability.
 6. **Strict TypeScript + strict templates**
 - Early error detection and safer refactoring.
 7. **Centralized toasts and notification center**
 - Consistent feedback mechanisms.
 8. **Chart component isolation**
 - Executive analytics visuals cleanly separated.
-

Risks/limitations and improvements

Observed limitations

1. **No token refresh strategy**
 - Session relies on static token in localStorage.
2. **Potential localStorage security concerns**
 - Standard SPA risk if XSS occurs.
3. **Some endpoint naming inconsistencies**
 - Mixed singular/plural and case (`User` vs `user` , `exceptionevent` fallback plural).
4. **Some duplicated role/dashboard mapping logic**
 - Appears in login and role guard.
5. **Some components use broad `any` responses**
 - Reduces type safety in places.
6. **Notification real-time is polling-based**
 - Not true push channel yet.
7. **Inconsistent status strings**
 - Multiple normalizers indicate backend/frontend naming drift.
8. **Minor dead fields**
 - Example: `errorMessage` declared in `ViewOrdersComponent` but unused.

Improvement recommendations

1. Add refresh-token flow + silent renew.
 2. Move role mappings/constants to one shared source (`login.model.ts` already has `ROLE_DASHBOARDS`).
 3. Standardize API route naming conventions with backend.
 4. Replace polling with SignalR/WebSocket for notifications.
 5. Introduce stronger response typing in all components.
 6. Add centralized status enum mapping utility.
 7. Add route-level preloading strategy for likely next screens.
 8. Expand test coverage for guards/resolvers/interceptors and critical forms.
-

Learning roadmap for a new developer (first-week study plan)

Day 1: App skeleton and runtime

- Read `main.ts` , `app.ts` , `app.config.ts` , `app.routes.ts` .
- Trace how standalone bootstrap and providers work.
- Follow one route from URL to rendered component.

Day 2: Security and session

- Study `authentication.service` , `login.service` , guards, interceptors.
- Verify login -> storage -> guard -> sidebar/header behavior.
- Map where role is checked and where redirect logic exists.

Day 3: Shared UX systems

- Review `main-layout` , `header` , `sidebar` , `toast-container` , `notification-center` .
- Understand role nav generation and active link matching.
- Trace toast publish/consume pattern.

Day 4: Core transactional domains

- Study `order.service` , `shipment.service` , `inventory.service` , `add-order` .
- Walk procurement and warehouse resolver effects on order types.
- Execute place-order and dispatch flows mentally/API-level.

Day 5: Admin and master data

- Study admin user/audit screens and network management triad.
- Understand parent orchestrators + list/form child component pattern.
- Review location/partner/lane/item/uom/inventory models.

Day 6: Planner operational control tower

- Study exception and resolution services/components.
- Understand filtering, CRUD, and owner-role assignment logic.

Day 7: Executive analytics and integration hardening

- Study KPI services and executive dashboard/reporting/chart components.
- Validate date filtering and chart lifecycle patterns.
- End by documenting one cross-feature improvement PR plan:
 - e.g., unify status vocabulary + role mapping constants.

Final technical takeaways

- This frontend is a **role-structured, API-driven, standalone Angular enterprise UI**.
- Architecture is already mature in separation of concerns:
 - route auth/context
 - domain services
 - DTO contracts
 - shared layout/feedback systems
- Main scaling opportunities are around:
 - session hardening
 - naming consistency
 - richer real-time events
 - deeper typing + automated tests

If you save this as a `.md` file, it is directly compatible with `md-to-pdf` conversion.

Complete source inventory (every corner checklist)

This appendix enumerates all detected core frontend source files in `src/app` so you can track full coverage while studying.

All component files (`src/app/components/**/*.component.ts`)

- `src/app/components/login/login.component.ts` — Authentication form and role redirect.
- `src/app/components/admin-dashboard/admin-dashboard.component.ts` — Admin route shell.
- `src/app/components/admin-dashboard/admin-dashboard-content/admin-dashboard-content.component.ts` — Admin overview cards and recent user metrics.

- `src/app/components/admin-dashboard/admin-manage-users/admin-manage-users.component.ts` — User list/search/edit.
- `src/app/components/admin-dashboard/admin-audit-logs/admin-audit-logs.component.ts` — Audit logs filter/export.
- `src/app/components/admin-dashboard/admin-create-user/admin-create-user.component.ts` — Admin-driven user creation.
- `src/app/components/admin-dashboard/admin-manage-network/admin-manage-network.component.ts` — Network management tab container.
- `src/app/components/admin-dashboard/admin-manage-network/manage-network-locations/manage-network-locations.component.ts` — Location CRUD orchestrator.
- `src/app/components/admin-dashboard/admin-manage-network/manage-network-locations/manage-network-locations-list/manage-locations-list.component.ts` — Location table/list actions.
- `src/app/components/admin-dashboard/admin-manage-network/manage-network-locations/manage-network-locations-form/manage-locations-form.component.ts` — Location create/edit form.
- `src/app/components/admin-dashboard/admin-manage-network/manage-network-lanes/manage-network-lanes.component.ts` — Lane CRUD orchestrator.
- `src/app/components/admin-dashboard/admin-manage-network/manage-network-lanes/manage-network-lanes-list/manage-network-lanes-list.component.ts` — Lane table/list actions.
- `src/app/components/admin-dashboard/admin-manage-network/manage-network-lanes/manage-network-lanes-form/manage-network-lanes-form.component.ts` — Lane create/edit form.
- `src/app/components/admin-dashboard/admin-manage-network/manage-network-partners/manage-network-partners.component.ts` — Partner CRUD orchestrator.
- `src/app/components/admin-dashboard/admin-manage-network/manage-network-partners/manage-network-partners-list/manage-partners-list.component.ts` — Partner list actions.
- `src/app/components/admin-dashboard/admin-manage-network/manage-network-partners/manage-network-partners-form/manage-partners-form.component.ts` — Partner create/edit form.
- `src/app/components/planner-dashboard/planner-dashboard.component.ts` — Planner route shell.
- `src/app/components/planner-dashboard/planner-dashboard-overview/planner-dashboard-overview.component.ts` — Planner overview KPIs.
- `src/app/components/planner-dashboard/planner-manage-exception-events/planner-manage-exception-events.component.ts` — Exception management.
- `src/app/components/planner-dashboard/planner-manage-resolution-actions/planner-manage-resolution-actions.component.ts` — Resolution action management.
- `src/app/components/logistics-dashboard/logistics-dashboard.component.ts` — Logistics route shell.
- `src/app/components/logistics-dashboard/logistics-dispatch-orders/logistics-dispatch-orders.component.ts` — Dispatch workflow.
- `src/app/components/logistics-dashboard/logistics-manage-shipments/logistics-manage-shipments.component.ts` — Shipment tracking and updates.
- `src/app/components/procurement-dashboard/procurement-dashboard.component.ts` — Procurement route shell.
- `src/app/components/procurement-dashboard/procurement-view-locations/procurement-view-locations.component.ts` — Read-only location view.
- `src/app/components/procurement-dashboard/procurement-view-partners/procurement-view-partners.component.ts` — Read-only partner view.
- `src/app/components/warehouse-dashboard/warehouse-dashboard.component.ts` — Warehouse route shell.
- `src/app/components/warehouse-dashboard/warehouse-manage-inventory/warehouse-manage-inventory.component.ts` — Inventory query/adjust flows.

- `src/app/components/warehouse-dashboard/warehouse-add-uom/warehouse-add-uom.component.ts` — UOM management.
- `src/app/components/warehouse-dashboard/warehouse-add-item/warehouse-add-item.component.ts` — Item management.
- `src/app/components/executive-dashboard/executive-dashboard.component.ts` — Executive parent dashboard and section routing.
- `src/app/components/executive-dashboard/executive-kpi-summary/executive-kpi-summary.component.ts` — KPI summary cards.
- `src/app/components/executive-dashboard/executive-kpi-trends/executive-kpi-trends.component.ts` — Trend filtering and chart orchestration.
- `src/app/components/executive-dashboard/executive-kpi-trends/performance-trends/performance-trends.component.ts` — Performance trend chart/panel.
- `src/app/components/executive-dashboard/executive-kpi-trends/top-inventory-turns/top-inventory-turns.component.ts` — Top inventory turns chart/panel.
- `src/app/components/executive-dashboard/executive-risks/executive-risks.component.ts` — Risk analytics panel.
- `src/app/components/executive-dashboard/executive-reports/executive-reports.component.ts` — Report list/generate/export.
- `src/app/components/shared/layout/main-layout/main-layout.component.ts` — Authenticated app frame.
- `src/app/components/shared/layout/header/header.component.ts` — Header identity, notifications, profile, logout.
- `src/app/components/shared/layout/sidebar/sidebar.component.ts` — Role-based navigation.
- `src/app/components/shared/toast-container/toast-container.component.ts` — Toast renderer.
- `src/app/components/shared/notification-center/notification-center.component.ts` — Notification inbox.
- `src/app/components/shared/view-profile/view-profile.component.ts` — Profile view.
- `src/app/components/shared/view-orders/view-orders.component.ts` — Generic orders list/filter/details.
- `src/app/components/shared/add-order/add-order.component.ts` — Generic order creation.
- `src/app/components/shared/not-found/not-found.component.ts` — Not-found page.

All services (`src/app/services/*.ts`)

- `src/app/services/authentication.service.ts`
- `src/app/services/login.service.ts`
- `src/app/services/user.service.ts`
- `src/app/services/manage-user.service.ts`
- `src/app/services/location.service.ts`
- `src/app/services/lane.service.ts`
- `src/app/services/partner.service.ts`
- `src/app/services/item.service.ts`
- `src/app/services/uom.service.ts`
- `src/app/services/inventory.service.ts`
- `src/app/services/order.service.ts`
- `src/app/services/shipment.service.ts`
- `src/app/services/exception-event.service.ts`
- `src/app/services/resolution-action.service.ts`
- `src/app/services/kpi.service.ts`
- `src/app/services/executive-dashboard.service.ts`

- `src/app/services/notification-event.service.ts`
- `src/app/services/order-context.service.ts`
- `src/app/services/toast.service.ts`

All models (`src/app/models/*.ts`)

- `src/app/models/enums.ts`
- `src/app/models/index.ts`
- `src/app/models/login.model.ts`
- `src/app/models/app-user.model.ts`
- `src/app/models/create-user.model.ts`
- `src/app/models/audit-log.model.ts`
- `src/app/models/location.model.ts`
- `src/app/models/lane.model.ts`
- `src/app/models/partner.model.ts`
- `src/app/models/uom.model.ts`
- `src/app/models/item.model.ts`
- `src/app/models/inventory.model.ts`
- `src/app/models/order.model.ts`
- `src/app/models/shipment.model.ts`
- `src/app/models/exception-event.model.ts`
- `src/app/models/resolution-action.model.ts`
- `src/app/models/kpi.model.ts`
- `src/app/models/notification-event.model.ts`

All guards (`src/app/guards/*.ts`)

- `src/app/guards/auth.guard.ts`
- `src/app/guards/role.guard.ts`
- `src/app/guards/order-context.resolver.ts`

All interceptors (`src/app/interceptors/*.ts`)

- `src/app/interceptors/auth.interceptor.ts`
- `src/app/interceptors/error.interceptor.ts`

Topic-wise quick notes (same format for all major topics)

1. Angular Application Bootstrapping

What is it?

Bootstrapping is the startup process where Angular loads the first root component and global providers.

How it works in this project:

```
// src/main.ts
import { bootstrapApplication } from '@angular/platform-browser';
import { registerLocaleData } from '@angular/common';
import localeEnIn from '@angular/common/locales/en-IN';
import { appConfig } from './app/app.config';
import { App } from './app/app';
```

```
registerLocaleData(localeEnIn);

bootstrapApplication(App, appConfig)
  .catch((err) => console.error(err));
```

```
// src/app/app.config.ts
export const appConfig: ApplicationConfig = {
  providers: [
    { provide: LOCALE_ID, useValue: 'en-IN' },
    { provide: DEFAULT_CURRENCY_CODE, useValue: 'INR' },
    provideZoneChangeDetection({ eventCoalescing: true }),
    provideRouter(routes),
    provideHttpClient(withInterceptors([authInterceptor, errorInterceptor]))
  ]
};
```

2. Creating a New Angular Application

What is it?

This is the Angular CLI workflow to scaffold a fresh Angular app.

How to create:

```
npm install -g @angular/cli
ng new TalentSphereApp
```

Typical generated structure:

```
src/
├─ app/           → Components, services, routes
├─ assets/        → Images, fonts
├─ index.html     → Root HTML file
├─ main.ts        → Entry point
└─ styles.css     → Global styles
```

3. Routing and Navigation

What is it?

Routing maps URLs to components. This project uses role-based nested routing.

How it works:

- Public route: `/login`
- Protected shell route: `/` with `MainLayoutComponent`
- Child routes per role (Admin, Planner, Logistics, Executive, Procurement, Warehouse)
- Fallback route to `/404`

```
// src/app/app.routes.ts (pattern)
{
  path: '',
  component: MainLayoutComponent,
  canActivate: [authGuard],
  children: [
    {
      path: 'planner-dashboard',
      canActivate: [RoleGuard],
      data: { roles: ['Planner'] },
      loadComponent: () => import('...').then(m => m.PlannerDashboardComponent)
    }
  ]
}
```

4. Authentication and Session Management

What is it?

Authentication validates users and stores login state. Session state is centralized in `AuthenticationService`.

How it works:

- Login API returns JWT token
- Frontend decodes token to extract role/user claims
- Stores `authToken` and `currentUser` in `localStorage`
- Maintains reactive user state with `BehaviorSubject`

```
// src/app/services/authentication.service.ts (concept)
setCurrentUser(user: CurrentUser, token: string): void {
  this.currentUserSubject.next(user);
  localStorage.setItem('authToken', token);
  localStorage.setItem('currentUser', JSON.stringify(user));
}
```

5. Guards and Access Control

What is it?

Guards protect routes before components load.

How it works in this app:

- `authGuard` : blocks unauthenticated users
- `RoleGuard` : blocks users without required route role
- Redirects unauthorized users to login or role dashboard

```
// src/app/guards/auth.guard.ts (concept)
if (!authService.isLoggedIn()) {
  router.navigate(['/login'], { queryParams: { returnUrl: state.url } });
}
```



```
    return false;
}
```

6. Route Resolvers (Order Context)

What is it?

Resolvers prepare contextual state before route activation.

How it works:

- `OrderContextResolver` => default order visibility
- `ProcurementOrderContextResolver` => hide/disable procurement-incompatible order types
- `WarehouseOrderContextResolver` => hide/disable warehouse-incompatible order types

7. HTTP Interceptors

What is it?

Interceptors are middleware for outgoing requests and incoming errors.

How it works:

- `authInterceptor` adds `Authorization` header for non-login API calls
- `errorInterceptor` maps technical HTTP errors to user-friendly `userMessage`

```
// src/app/interceptors/auth.interceptor.ts (concept)
const clonedReq = req.clone({
  setHeaders: { Authorization: `Bearer ${token}` }
});
```

8. Layout System (Root, Header, Sidebar)

What is it?

Layout components provide common UI frame for authenticated pages.

How it works:

- `App` => global toast container + root outlet
- `MainLayoutComponent` => header + sidebar + feature outlet
- `HeaderComponent` => profile + logout + notifications
- `SidebarComponent` => role-based dynamic menu

9. Toast and Notification Flow

What is it?

Two user feedback channels:

- Toasts: lightweight action/status messages
- Notification center: inbox-like events with unread counts

How it works:

- `ToastService` publishes toast events
 - `ToastContainerComponent` renders and dismisses
 - `NotificationCenterComponent` loads notifications, polls every 30 sec, supports read/delete actions
-

10. Component Architecture (Feature + Shared)

What is it?

Components are organized by business role and shared utilities.

How it works:

- Feature folders: `admin-dashboard` , `planner-dashboard` , `logistics-dashboard` , `executive-dashboard` , `procurement-dashboard` , `warehouse-dashboard`
 - Shared folder: layout, add-order, view-orders, profile, notifications, toast, not-found
 - Standalone components are lazy-loaded for many routes
-

11. Service Layer Architecture

What is it?

Services isolate API calls and reusable business logic.

How it works:

- Domain services map to backend resources: users, locations, lanes, partners, inventory, items, uoms, orders, shipments, exceptions, resolutions, KPI, notifications
 - Components consume services and bind results to view state
 - Some services manage UI context/state (`OrderContextService` , `ToastService`)
-

12. Model/DTO Layer

What is it?

Models define TypeScript contracts for request/response payloads.

How it works:

- Each domain has DTO files in `src/app/models`
 - `index.ts` provides barrel exports
 - `enums.ts` centralizes shared enum values
-

13. End-to-End Business Flows

What is it?

Business flows combine routing, UI, services, and APIs.

Key flows covered in this app:

- Login and role-based redirect
- Add order (procurement/warehouse constraints)
- Dispatch and shipment lifecycle
- Inventory adjust/create
- Exception and resolution management
- Executive KPI trend/risk/report exploration

- Notification read/clear lifecycle
-

14. Configuration and Build System

What is it?

Project-level setup for compile, serve, and test.

How it works:

- `angular.json` defines build/serve/test targets
 - `tsconfig.json` enables strict mode and path aliases
 - `app-settings.ts` stores backend endpoint base URL
-

15. Codebase Strengths and Improvement Opportunities

What is it?

A quick engineering quality review.

Strengths:

- Clear role-based architecture
- Strong separation: components/services/models/guards/interceptors
- Consistent standalone Angular pattern

Improvements:

- Add refresh-token flow
- Replace polling with real-time push for notifications
- Normalize status naming across frontend/backend
- Increase unit/integration test coverage